

User Form — Workspace Access Visibility Audit

Investigating the report that the User Create/Edit form no longer shows the Workspace Access capability group after Governance G2. Read-only audit against the committed source on branch `feature/mobile-ui-pass-1`, tip `16d93c3`, plus a live query against the connected database. No code modifications.

HEADLINE — In the committed code path, the User form's `UserCapabilityPicker` WILL render Workspace Access. The component does not pass `groupKeys`, the server returns `INSPECTION` and `MAINTENANCE`, the `isAssignableCapability` predicate accepts both, and `groupCapabilities()` places them in the Workspace Access bucket. If the deployed UI is missing the group, the cause is operational (stale bundle, wrong form, viewport), not source-level. Verification steps in §6.

1. Is Workspace Access intentionally hidden on the User form?

No. No code path in the User Create/Edit flow hides the Workspace Access fieldset. The only per-context allow-list in the admin web today is `MAINHEAD_PICKER_GROUP_KEYS = ["ASSET_DOMAIN"]`, applied exclusively in the `MAINHEAD` branch of `EnterpriseFormModal` (`enterprise-list-client.tsx:858`). It is not imported, referenced, or applied anywhere in `users-client.tsx`.

Grep across `users-client.tsx` for `INSPECTION`, `MAINTENANCE`, `WORKSPACE`, `workspace`, and `groupKeys` returns **zero matches**. There is no string literal or branch in the User form that suppresses workspace codes.

2. Was Workspace Access accidentally removed during Governance G2?

No. Governance G2 (commit `16d93c3` — *"Implement Governance G2 MAINHEAD capability restriction"*) did not modify `users-client.tsx`. Verified by `git show --stat 16d93c3` — the change set lists `enterprise-list-client.tsx`, `site-visit-detail-client.tsx`, `site-visits-client.tsx`, `capability-groups.ts`, `canonical-capabilities.ts`, `enterprise.service.ts`, `users.service.ts`, two site-visit DTO/service files, but does **not** list `users-client.tsx`.

G2 changes that touched the resolver-side and `MAINHEAD` picker were intentional and scoped. The User capability picker was left as-is, by design: per the original G2 requirements (§1 of the brief), the User form must remain unchanged.

3. Can users still be assigned `INSPECTION` or `MAINTENANCE` through the UI?

Yes — in the source as committed. The User Create/Edit form mounts `UserCapabilityPicker` (`users-client.tsx:589`). The picker:

- Calls `options.capabilities.filter(isAssignableCapability)`. `isAssignableCapability` drops rows that are explicitly `isActive=false` or whose code is outside the canonical allow-list. `INSPECTION` and `MAINTENANCE` are both in `CANONICAL_CAPABILITY_CODES` and pass.
- Calls `groupCapabilities(assignable)` — buckets each capability by `capabilityGroupFor(code)`. `INSPECTION` and `MAINTENANCE` both bucket to `WORKSPACE`,

populating the Workspace Access group.

- Does **not** pass `groupKeys`. The optional per-context filter exists only on `CapabilityPicker` in `enterprise-list-client.tsx`; the User form uses a separate component (`UserCapabilityPicker`) that has no analogous prop.
- Renders all three groups in canonical order: Workspace Access, Governance & Reporting, Asset Domains.

Server-side cross-check (live DB)

Ran the exact same query the API uses for `GET /enterprise/options.capabilities` against the connected DB:

```
prisma.capability.findMany({
  where: { isActive: true, code: { in: [...CANONICAL_CAPABILITY_CODES] } },
  orderBy: [{ isActive: 'desc' }, { name: 'asc' }],
  select: { id, name, code, description, isActive },
})
→ count: 12, includes INSPECTION (id ...011) and MAINTENANCE (id ...009),
  both isActive=true.
```

The server is returning both workspace codes today.

4. Could new users created today lose workspace visibility because Workspace Access cannot be assigned?

Per the source as committed: No. Admins creating new users via the User form can tick INSPECTION or MAINTENANCE under Workspace Access; the form posts those capability IDs to the API; the User-direct UserCapability rows are written; the effective capability resolver continues to include User-sourced rows. Mobile workspace gating then opens the Inspection or Maintenance workspace as before.

If the deployed UI in fact hides Workspace Access (a real but unverifiable claim from the source repo alone), the operational impact would be:

- **New users created via the User form would not receive workspace capabilities at the User level.** No checkbox to grant INSPECTION or MAINTENANCE in the form.
- **Per Governance G2, MAINHEAD capabilities no longer grant authority.** So the only remaining grant paths are Team / Branch / Organization — all of which require additional admin steps outside the User Create/Edit flow.
- **Result:** a new user with no Team / Branch / Organization workspace assignment would log into mobile and see no workspaces. The mobile workspace resolver gates on INSPECTION / MAINTENANCE in the effective set (`operationalWorkspace.ts:10–11, 64, 68`); without any of those four sources, the user can't enter either workspace.
- ADMIN users would still see both workspaces via the resolver short-circuit (`users.service.ts:488–505`), so the issue is invisible to admins testing with their own accounts.

5. Component and filtering logic responsible

Layer	Location	What it does
Component	apps/admin-web/src/components/users-client.tsx lines 236-298 – UserCapabilityPicker line 589 – render site inside UserFormModal	Renders capability checkboxes for the User Create/Edit modal. Independent of the enterprise-list-client picker.
Server filter	apps/api/src/enterprise/enterprise.service.ts lines 428-447 – capability.findMany inside getOptions	Server-side allow-list applied to /enterprise/options. Returns only canonical + active. INSPECTION + MAINTENANCE both included today.
Client allow-list	apps/admin-web/src/lib/capability-groups.ts lines 60-91 – CANONICAL_CAPABILITY_CODES + isAssignableCapability	Defense-in-depth filter. Drops rows that are explicitly inactive or whose code is outside the canonical set. Accepts INSPECTION + MAINTENANCE.
Group bucketing	apps/admin-web/src/lib/capability-groups.ts lines 30-33, 107-121, 152-186 – WORKSPACE_CODES, capabilityGroupFor, groupCapabilities	Maps codes to one of three buckets and returns groups in canonical order (Workspace Access → Governance & Reporting → Asset Domains). Empty buckets are dropped — but with INSPECTION + MAINTENANCE present, the WORKSPACE bucket is non-empty and renders.
Per-context restriction	apps/admin-web/src/lib/capability-groups.ts lines 137-145 – MAINHEAD_PICKER_GROUP_KEYS	Defined and exported. Imported and used only by enterprise-list-client.tsx:858 (the MAINHEAD branch of EnterpriseFormModal). Not imported by users-client.tsx. This is the only per-group restriction in the codebase.

6. Operational verification steps (recommended before any code action)

Because the source as committed should render Workspace Access on the User form, the report of its absence is most likely an operational artefact. Recommend these checks before treating it as a source-level bug:

- **Hard refresh of the admin browser tab** (Ctrl/Cmd+Shift+R). Next.js + Turbopack chunk hashes can drift between deploys; a cached chunk loaded before the last admin-web rebuild may render an older version of the picker.
- **Confirm the deployed build matches the commit.** `git rev-parse HEAD` on the build host should equal 16d93c3 (or a descendant). If a stale dev server is running, restart it.
- **Confirm the modal under inspection is the User modal**, not the MAINHEAD modal. The MAINHEAD modal correctly shows only Asset Domains; visual layout is similar (header says MAINHEAD vs User; only the User modal contains the Email field and the role dropdown).
- **Network tab:** open browser devtools, open the User Create modal, observe the GET /enterprise/options response payload, and verify capabilities includes objects with code: "INSPECTION" and code: "MAINTENANCE". The dev DB confirms the server returns them; if the

response on staging differs, that points to a different DB or a different deployed API.

- **Inspect the rendered DOM.** Search for the legend text “Workspace Access”. If the legend is present but the checkboxes are missing, the issue is the capability list (server response); if the legend itself is missing, the issue is `groupCapabilities()` dropping an empty bucket — which only happens when no INSPECTION / MAINTENANCE rows survive the filters.
- **Check the live capability rows.** If INSPECTION or MAINTENANCE were deactivated in the catalogue page (`/capabilities`) by an ops action — `isActive=false` — the server-side filter `isActive:true` would exclude them and the WORKSPACE bucket would be empty, so `groupCapabilities()` would drop the Workspace Access legend entirely. **The most likely real-world cause if the deployment is current.**

7. Bottom line

- **Source verdict:** Workspace Access is not hidden on the User form; G2 did not remove it; the picker continues to support assigning INSPECTION / MAINTENANCE; new users created via the form can receive workspace authority via the User direct grant path.
- **Most likely operational cause for the observed UI:** either a stale client bundle, or one of the two workspace capability rows was deactivated in the catalogue page (which causes `groupCapabilities()` to drop the empty Workspace Access bucket entirely).
- **No action required in source.** The per-context allow-list infrastructure in `capability-groups.ts` is correctly scoped to the MAINHEAD picker only. If after the verification steps the issue persists, the next step is to look at the `capabilities` array in the actual `GET /enterprise/options` response on the affected environment, not the client code.